

Economic Invariant Analysis of a Single Transaction DeFi Flash Loan Model

Shorya Jaiswal

Symbiosis International University (Symbiosis Artificial Intelligence Institute)

shoryajkt@gmail.com

Abstract—A flash loan can temporarily distort the price seen by a lending protocol even when the trading pool returns to its initial reserves. This paper studies a bounded, fee-free constant-product automated market maker (AMM) coupled to a lender that accepts a manipulated spot price. The attacker pledges pre-owned collateral, borrows against the distorted price, reverses the AMM trade, repays the flash loan, and forfeits collateral under an explicit non-recourse settlement assumption. We derive an exact finite-candidate test for profitable extraction, a sufficient reserve-depth threshold, and conditional defenses based on bounded oracle prices and dynamic loan-to-value ratios. A Python/Z3 model searches for counterexamples; the analytical test agrees with all 90 synthetic parameter settings. A separate counterfactual uses rounded AMM reserves reported for a February 2020 bZx transaction; it is not a replay of that transaction. An auxiliary conservation experiment measures solver behavior for 1–32 distinct symbolic pools. These model-level results do not establish security of deployed contracts.

Index Terms—decentralized finance; flash loans; economic security; constant-product AMM; satisfiability modulo theories; oracle manipulation.

I. INTRODUCTION

Flash loans make capital available within a single atomic transaction. Atomic repayment does not itself guarantee that every connected protocol is economically safe. A price-sensitive lending protocol can transfer funds based on a transiently distorted AMM price even after the AMM reserves are restored. Earlier work studied profitable DeFi transaction discovery and mechanized economic security [1–4]. This paper isolates one lending composition and asks what can be proved about it under explicit settlement assumptions.

The paper makes three contributions:

- It gives a complete wallet, reserve, debt, fee, and collateral accounting for a fixed atomic attack schedule. This makes positive extraction a testable property rather than an inference from price movement alone.
- It derives a finite candidate characterization of maximal extraction, plus sufficient oracle, loan-to-value, and reserve-depth conditions that exclude positive extraction in the model.
- It provides a Python/Z3 implementation and exact rational witness replay, and checks agreement between the solver and analytical criterion over a controlled synthetic parameter grid.

The analytical result is the main contribution for this restricted model. SMT is not required to decide its closed-form profit function. It serves as an independent encoding check and a route toward more complex transition systems whose profit objective may lack this simple reduction. The paper does not claim that using SMT for DeFi is new [2–4].

Research questions

RQ1: Can the AMM return to its initial reserves while the lender suffers positive net extraction? RQ2: Which modeled oracle and lending constraints exclude that extraction? RQ3: Do Z3 decisions agree with the analytical criterion for every tested parameter setting? Section 8 answers each question directly.

II. THREAT MODEL AND NOTATION

Token A is collateral and token B is the accounting unit. The AMM begins with positive reserves x units of A and y units of B, with reference price $p_0 = y/x$. The attacker already owns C units of A, worth $V = Cp_0$ at that reference price. It may flash-borrow f units of B, $0 < f \leq F$, at fee rate $r \geq 0$. The lender has cash T and a loan-to-value (LTV) limit $0 < L < 1$. The attacker can pledge the pre-owned C units, borrow b units of B, reverse its AMM trade, and repay the flash lender. Ordinary lending debt remains; the model assumes the attacker can forfeit all C units without recourse to its remaining wallet. That assumption is necessary for the extraction claim.

Table 1. Symbols and baseline synthetic values.

Symbol	Meaning	Baseline
x, y	AMM reserve quantities	1000, 1000
C, V	Pre-owned collateral and reference value	100 A; 100 B
L, r	LTV limit and flash fee rate	0.75; 0.0009
F, T	Maximum flash loan and lender cash	1000 B; 400 B
f, b	Chosen flash loan and lending borrow	Decision variables

The fixed schedule does not enumerate arbitrary contract calls. The model omits AMM swap fees, gas, token rebases, other market participants, cross-block effects, contract storage semantics, and liquidation settlement. All numeric token amounts used below are synthetic unless specifically attributed to a historical source.

III. STATE TRANSITIONS AND EXTRACTION

A flash input of B buys $a = xf/(y+f)$ units of A. The first swap gives reserves $x_1 = xy/(y+f)$ and $y_1 = y+f$. Since collateral A is priced in B, its post-swap AMM spot price rises to $p_1 = p_0(1+f/y)^2$. An input of A would move that price in the opposite direction.

$$y_1 = y + f; \quad x_1 = xy/(y + f); \quad a = xf/(y + f) \quad (1)$$

$$p_1 = p_0(1 + f/y)^2; \quad 0 \leq b \leq \min(T, CLp_1) \quad (2)$$

The attacker pledges its pre-owned C units, separately from the newly acquired a units. Selling all a back to the same fee-free pool returns exactly f units of B and restores both AMM reserves. After repaying $f(1+r)$, the wallet contains $b-rf$ units of B . The flash lender receives fee rf ; the lending protocol holds the forfeited C units and an outstanding claim for debt b . The attacker's gain relative to its original collateral value is:

$$\Pi = b - rf - V \quad (3)$$

A positive Π therefore requires that the borrow exceed the collateral opportunity cost and flash fee. If ordinary debt must be repaid from the wallet, this fixed attack schedule has no such gain. The model does not treat the unpaid borrowing as a free asset.

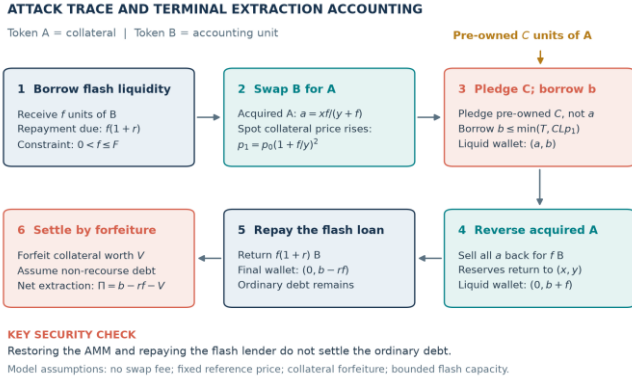


Figure 1. The fixed model schedule. The diagram distinguishes pre-owned collateral from the tokens acquired in the AMM swap.

IV. ANALYTICAL RESULTS

A. Exact profit criterion

For fixed f , the borrower chooses the largest permitted b . With $A_0 = VL$, the maximal net extraction is the capped function:

$$g(f) = \min\{T, A_0(1 + f/y)^2\} - V - rf, \quad 0 < f \leq F \quad (4)$$

On its uncapped branch, $g''(f) = 2A_0/y^2 > 0$. Thus any stationary point is a minimum, not a profit maximum. Once lending cash T binds, the slope becomes $-r \leq 0$. The only candidate values needed for the global supremum are $g(0)$ as an excluded endpoint limit, $g(F)$, and the cash-cap kink when it lies in $[0, F]$:

$$f_c = y[\sqrt{(T/A_0)} - 1] \quad (5)$$

Theorem 1 (finite candidates). For $C > 0$, the largest value among those admissible candidates is positive if and only if a feasible profitable trace exists. If $T < A_0$, the cash cap already binds at $f = 0$ and the endpoints suffice. Proof. Convexity places the maximum of each closed uncapped subinterval at an endpoint. On the capped interval g is nonincreasing. Splitting $[0, F]$ at f_c yields the candidate list. Because $g(0) \leq V(L-1) < 0$, any positive maximum occurs at an allowed nonzero f . For $C = 0$, $b = 0$ and Π cannot be positive.

For $x = y = 1000$, $C = 100$, $L = 0.75$, $r = 0.0009$, $F = 1000$, and $T = 400$, $f = 1000$ gives $a = 500$, $p_1 = 4$, $b = 300$, and $\Pi = 199.1$ B . These normalized values are illustrative, not bZx reserves or exploit proceeds.

B. Conditional defenses and depth

Proposition 2 (bounded oracle). Suppose the lender uses a price satisfying $0 < p_{\text{oracle}} \leq p_0(1+\epsilon)$. If $L(1+\epsilon) \leq 1$, then $b \leq CLp_0(1+\epsilon) \leq V$ and $\Pi \leq -rf \leq 0$. The proof is conditional on the price bound; the software does not verify a deployed time-weighted oracle.

A rational dynamic policy $L(f) = L_0/(1+f/y)^2$ cancels the modeled price distortion and yields $b \leq VL_0 < V$ when $L_0 < 1$. An exponential policy $L(f) = L_0e^{-\lambda f}$ also suffices when $\lambda \geq 2/y$, using $\log(1+u) \leq u$. The exponential condition is an analytical result and is not encoded exactly in the polynomial Z3 query.

Holding p_0 , V , L , r , and F fixed, ignoring the lending cash cap gives a conservative depth threshold:

$$y_{\text{safe}} = F/[\sqrt{((V + rF)/(VL))} - 1]; \quad k_{\text{safe}} = y_{\text{safe}}^2/p_0 \quad (6)$$

If $y \geq y_{\text{safe}}$, the uncapped value at F and the zero-endpoint limit are nonpositive, so convexity excludes positive extraction for every $f \leq F$. With a binding cash cap, this bound is sufficient but may be unnecessarily large. It is not a universal minimum liquidity function of total value locked alone.

V. IMPLEMENTATION AND REPRODUCIBILITY

The Python project separates AMM transitions, lending guards, Z3 encoding, analytical candidate evaluation, exact rational witness replay, and experiment drivers. The core query asks whether there exists a feasible f and b with $\Pi > 0$. Z3 checks this negation of economic safety in nonlinear real arithmetic; the code records SAT, UNSAT, or UNKNOWN. SAT witnesses are independently replayed with Python Fraction arithmetic. UNSAT relies on the encoding and Z3; no independently checked proof certificate or contract-to-model refinement is produced.

The supplied environment was verified on 26 September 2026: Python 3.14.7, Z3 5.1.0, and SymPy 1.14.0. These match the saved run metadata. Z3 5.1.0 is therefore the installed solver version, rather than a transcribed table error. The optional unsigned 256-bit module checks a concrete arithmetic trace only; its SAT result is not a profitable exploit witness.

A. Code artifact and implementation

The public repository <https://github.com/shoryaj/crypto-paper> contains the full source, tests, and saved outputs. Table 2 maps the mathematical steps to files a reader can inspect. The core is Python code, not a deployed smart contract.

Table 2. Where the executable parts of the paper are implemented.

Repository path	Purpose	Paper connection
src/core/amm_model.py	Encodes a fee-free constant-product swap	Reserve update in Eq. (1)
src/core/verifier.py	Builds swap, oracle, lending, profit, and Z3 constraints; replays SAT witnesses exactly	Eqs. (2)–(3) and invariant query
src/core/analysis.py	Computes finite candidate values and liquidity bound	Theorem 1 and Eq. (6)
src/utis/benchmark.py	Runs the 90-setting synthetic grid and exports SMT-LIB	Section 7.1

Repository path	Purpose	Paper connection
	queries	
src/utls/reviewer_extensions.py	Runs published-reserve counterfactual and pool scaling experiment	Sections 7.3–7.4
tests/test_verification.py	Exercises economic decisions, controls, and model accounting	Regression checks

The AMM primitive implements the constant-product invariant as a polynomial equality. This avoids division by an unconstrained symbolic expression; positive reserve guards preserve the intended domain. The following lines are from `src/core/amm_model.py`:

```
solver.add(reserve_in > 0, reserve_out > 0, amount >= 0,
           output >= 0, output < reserve_out,
           (reserve_in + amount) * (reserve_out - output)
           == reserve_in * reserve_out)
```

In `src/core/verifier.py`, the lending constraint produces borrow; then the program accounts for the AMM reversal, flash-loan repayment, and original collateral value. A positive profit is the counterexample sought by Z3:

```
self.borrow = borrowing(self.solver, self.c, oracle, ltv, self.T)
self.wallet_final = self.borrow + self.b_return - self.f * (1 + self.r)
self.profit = self.wallet_final - self.c * self.p0
self.solver.add(self.wallet_final >= 0)
```

The independent analytical path in `src/core/analysis.py` evaluates $f = 0$, $f = F$, and any admissible cash-cap kink. It compares their exact symbolic values with the Z3 decision. The benchmark stores each solver query under `results/queries` and records outcomes in `results/sweep.csv` and `results/sweep.json`. The tests run with `python -m pytest -q`; the recorded run passed 31 tests. Readers can rerun the benchmark with `python -m src.utls.benchmark` and the added case studies with `python -m src.utls.reviewer_extensions`. These short excerpts show the coding logic without using screenshots as scientific evidence; the complete functions remain in the repository.

HOW THE VERIFIER CHECKS ECONOMIC SAFETY

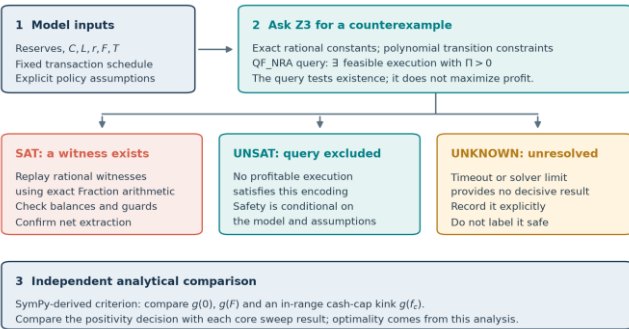


Figure 2. Verification path. Analytical classification and exact replay are separate checks; UNKNOWN is unresolved.

VI. CASE STUDIES AND RELATION TO PRIOR TOOLS

The bZx-inspired example isolates transient spot-price inflation before lending [1]. A reserve-calibrated counterfactual based on the historical Uniswap sUSD/ETH pool is reported in Section 7.3. The Harvest proxy uses two independent constant-product pools to perturb a vault valuation, then checks fee-net share redemption; the corrected

proxy mints shares from true assets [12]. The Euler proxy asks whether a donation can cause an unhealthy collateral position; a post-donation health check blocks that transition [13]. These simplified examples capture different failure mechanisms. They are not historical transaction replays, and the Euler SAT result concerns health reachability rather than profit.

Table 3. Scope of this artifact and established systems.

System	Published focus	Comparison to this paper
DeFiPoser [2]	Profitable transaction discovery across DeFi actions	Broader action search; includes an economic bZx case.
Clockwork Finance [3]	Mechanized economic properties of composed contracts	Broader formal framework and richer compositions.
FlashSyn [4]	Counterexample-guided attack synthesis from approximations	Historical benchmark synthesis beyond this fixed schedule.
FORAY [5]	Token-flow attack sketches and SMT completion	Automated path synthesis; this paper assumes a fixed path.
This artifact	Exact closed-form criterion for one capped lending composition	Transparent settlement accounting and a conditional depth bound.

FlashSyn provides an executable research artifact with a bZx1 benchmark [14]. Its authors specify a Linux/Docker setup and note that the artifact is not supported on Windows. The present experiment uses a preselected, fee-free swap–lend–reverse schedule and a non-recourse settlement rule; FlashSyn synthesizes action sequences against its own benchmark semantics. Thus a direct agreement or speed comparison would require porting one common model and state into both tools. The comparison here is methodological and reproducibility-focused, not a matched runtime claim. Formal AMM models and oracle research likewise predate this work [6–8].

VII. EVALUATION

A. Experimental grid

The saved benchmark varies equal reserve depths $x = y$ in $\{100, 1000, 10000\}$, LTV values from 0.50 through 0.95 in steps of 0.05, and flash fees 0.0009, 0.0015, and 0.003. The remaining values are fixed at $C = 100$, $F = 1000$, and $T = 400$. This gives 90 distinct settings and three sequential Z3 calls per setting. The analytical candidate method classifies all 90 settings; its decisions agree with every Z3 result. All recorded core SAT witnesses replay exactly. Sequential repeats are reproducibility checks, not independent samples or additional protocol cases.

Table 4. Recorded model decisions and solver-check times.

Equal reserves	SAT settings	UNSAT settings	SAT calls	UNSAT calls	Median ms
100	30	0	90	0	20.606
1000	30	0	90	0	20.785
10000	8	22	24	66	2.233
Total	68	22	204	66	—

The 204 SAT and 66 UNSAT totals count solver calls, whereas 68 and 22 count distinct settings. All 270 checks were decisive. Timing covers solver.check only, excluding encoding, analysis, and exact replay. RSS snapshots in the raw results are process-wide and do not measure peak solver overhead.

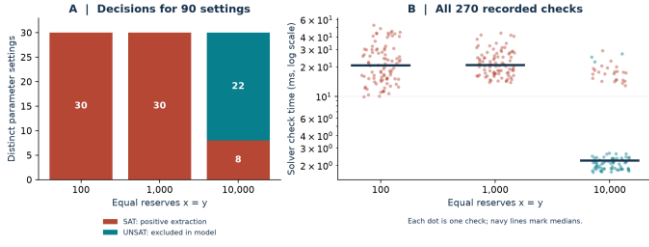


Figure 3. Recorded decisions by distinct setting and distribution of all 270 solver-check times.

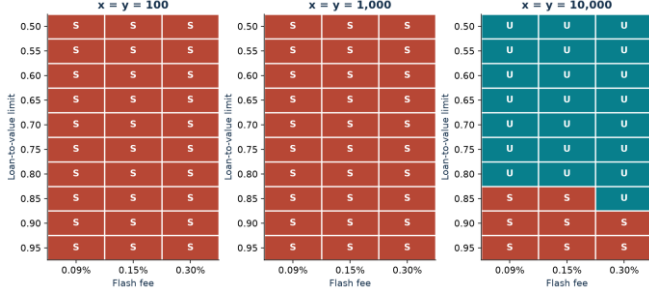


Figure 4. The 90 recorded decisions. Each cell is one setting, with three agreeing Z3 repeats.

B. Analytical baseline and example witnesses

For this restricted model, the closed-form candidate rule is a complete decision baseline: it checks at most three candidate amounts instead of searching the nonlinear constraint system. The comparison yields 90/90 agreement at the level of distinct settings, and 270/270 agreement across repeated solver calls. This establishes internal consistency, not independent correctness of the model or superior performance over DeFiPoser, Clockwork Finance, FlashSyn, or FORAY.

Table 5. Decision targets for selected controls.

Encoded property	Result	Meaning
Spot-price core extraction	SAT	Positive model extraction exists.
Tight oracle bound $\varepsilon = 0.1$	UNSAT	Positive core extraction excluded under assumed bound.
Loose oracle bound $\varepsilon = 1$	SAT	An allowed abstract oracle price permits extraction.
Rational dynamic LTV	UNSAT	Positive core extraction excluded under this policy.
Harvest vault proxy / corrected proxy	SAT / UNSAT	Fee-net proxy profit exists / is excluded.
Euler donation / guarded donation	SAT / UNSAT	Unhealthy state reachable / guarded.

One saved spot witness uses $f = 156$, $b = 1603/16$ and $\Pi = 471/10000 = 0.0471$ B. It proves satisfiability but is not an optimum. The analytical baseline example at $f = 1000$ yields 199.1 B. This difference is expected because a SAT solver needs only one profitable point.

ANALYTICAL PROFIT ENVELOPE UNDER FIXED FLASH CAPACITY
 $g(f) = \min \{T, VL(1 + f/y)^2\} - V - rf$

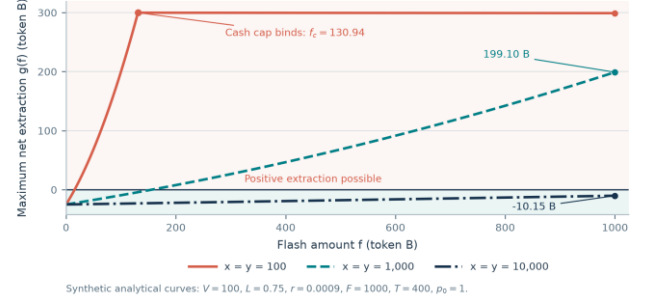


Figure 5. Analytical profit curves for fixed $V = 100$, $L = 0.75$, $r = 0.0009$, $F = 1000$, and $T = 400$. These are computed from the formula, not new solver measurements.

C. Case calibrated to published historical reserves

Qin et al. report that the Uniswap sUSD/ETH pool held 243,441.12 sUSD and 879.76 ETH immediately before the February 2020 bZx oracle manipulation; 540 ETH was sent to that pool, after which its reported reserves were 151,021.42 sUSD and 1,419.76 ETH [1, Fig. 4]. These are rounded published figures, not values independently fetched from an archive node. We set x and y to the reported starting reserves and F to the reported 540 ETH pool input. The hypothetical lender instead uses pre-owned $C = 100,000$ sUSD, $L = 0.75$, $T = 500$ ETH, and $r = 0.0009$. Those four quantities are scenario assumptions, not historical bZx parameters.

Table 6. Published pool reserves and a fee-free model counterfactual.

Quantity	Published bZx pool [1]	This model
Initial sUSD / ETH	243,441.12 / 879.76	Same starting reserves
Input to pool	540 ETH	$F = 540$ ETH
sUSD after input	151,021.42	150,849.27
sUSD paid out	92,419.70	92,591.85
Collateral, LTV, cash, flash fee	Different historical mechanism	100,000 sUSD; 0.75; 500 ETH; 0.0009
Modeled gain at $f = 540$	Not comparable to historical proceeds	138.129 ETH

The fee-free formula predicts 150,849.27 sUSD after the input, 172.15 sUSD below the published reserve because it omits the historical swap fee. The modeled spot-price multiplier is 2.6044. Under the hypothetical lender settings, the collateral reference value is 361.385 ETH and the 500 ETH cash cap yields $g(540) = 138.129$ ETH. Z3 returns SAT with an exactly replayed witness; the analytical rule agrees. This tests sensitivity to historical pool depth, not actual bZx profit. Inputs and outputs are saved in `results/reviewer_extensions/historical_reserve_case.json`.

D. Structural scaling of the auxiliary pool model

To vary model size rather than merely repeat a fixed query, we generated 1, 2, 4, 8, 16, and 32 distinct pools with one symbolic A-to-B swap per pool. Initial pool reserves are 1000 A and 1000 B; the wallet begins with 1000 A. Each query first checks feasibility, then asks for a violation of total token conservation. Three fresh queries were run per size on Windows with Python 3.12.14 and Z3 5.1.0; Table 7 reports median end-to-end check times, including both solver checks

but excluding model construction. All cases were feasible and the conservation counterexample was UNSAT.

Table 7. Auxiliary DAG structural scaling; three fresh runs per size.

Pools / swaps	Assertions	Median encoding ms	Median checks ms
1	11	1.628	6.138
2	20	2.481	5.923
4	38	4.416	5.142
8	74	5.872	7.415
16	146	15.612	10.009
32	290	28.956	14.377

These small, independent swaps exercise only the auxiliary conservation encoding. They do not enlarge the core lender attack schedule, synthesize paths, or establish asymptotic scalability for realistic DeFi compositions. Raw per-run measurements and the generator are in `results/reviewer_extensions/structural_scaling.json` and `src/utls/reviewer_extensions.py`.

VIII. DISCUSSION AND ANSWERS TO THE RESEARCH QUESTIONS

RQ1: Yes, in the specified model. A fee-free reverse trade restores x and y while the lender retains a debt claim against collateral whose reference value can be smaller than the borrowed amount. The positive 199.1 B example shows this explicitly. RQ2: A sufficiently tight oracle upper bound, the rational dynamic LTV policy, or sufficient reserve depth excludes positive extraction under their stated assumptions. A loose oracle bound does not. RQ3: Yes, all 90 distinct settings and 270 recorded Z3 calls agree with the exact candidate criterion, and each core SAT witness was replayed with rational arithmetic.

These answers do not establish exploit likelihood. The synthetic grid was chosen to exercise the model, not to sample deployed protocols. The reserve-calibrated counterfactual tests the model at one published historical pool depth, while Section 7.4 bounds what the auxiliary scaling experiment measures. A true historical transaction replay would require exact block state, oracle implementation, reserves, fees, liquidation behavior, and contract traces.

IX. LIMITATIONS AND VALIDITY

The model fixes a single transaction schedule and assumes non-recourse collateral forfeiture. It omits swap fees, gas, fee-on-transfer tokens, concurrent arbitrage, cross-block oracle behavior, and a deployed contract semantics. The auxiliary DAG module checks conservation for a finite topological list of distinct-pool swaps, but is not an exhaustive lending/liquidation composer. The case proxies should not be interpreted as recreations of bZx, Harvest, or Euler losses. Z3 UNSAT is conditional on the encoding; exact replay checks SAT witnesses but does not validate the mapping from real contracts to the abstraction.

The historical calibration uses rounded reserve figures from a published analysis rather than an archive-node extraction, and the remaining lender parameters are counterfactual. A matched executable comparison against another DeFi tool

remains open: the available FlashSyn artifact requires a Linux/Docker environment and implements different action and settlement semantics [14]. The auxiliary scaling experiment is not an attack-synthesis benchmark. A repository URL is not an archival DOI; a DOI requires a separately deposited release.

X. CONCLUSION

Explicit accounting makes an oracle-driven lending failure checkable even when the AMM itself finishes at its starting reserves. In the fixed capped model, the maximum extraction is determined by a small finite set of candidate flash amounts. Conditional oracle, dynamic LTV, and depth bounds can exclude positive extraction. The implementation corroborates those classifications across 90 synthetic settings and a reserve-calibrated counterfactual. Extending the result to deployed systems requires a historical transaction replay, contract-level refinement, and matched comparison with existing tools.

ACKNOWLEDGMENTS AND DECLARATIONS

AI assistance: OpenAI Codex and Anthropic Claude were used to assist with manuscript drafting and editing, portions of the Python prototype, figure preparation, and document formatting.

Data and code availability: Project code, synthetic results, and the manuscript are publicly available at <https://github.com/shoryaj/crypto-paper>. No archival DOI has been assigned.

Funding: The author received no funding or financial support for this work.

Conflicts of interest: The author declares no conflicts of interest.

REFERENCES

- [1] K. Qin, L. Zhou, B. Livshits, and A. Gervais, “Attacking the DeFi Ecosystem with Flash Loans for Fun and Profit,” *Financial Cryptography and Data Security*, 2021, pp. 3–32. https://doi.org/10.1007/978-3-662-64322-8_1
- [2] L. Zhou, K. Qin, A. Cully, B. Livshits, and A. Gervais, “On the Just-In-Time Discovery of Profit-Generating Transactions in DeFi Protocols,” *IEEE Symposium on Security and Privacy*, 2021, pp. 919–936. <https://doi.org/10.1109/SP40001.2021.00113>
- [3] K. Babel, P. Daian, M. Kelkar, and A. Juels, “Clockwork Finance: Automated Analysis of Economic Security in Smart Contracts,” *IEEE Symposium on Security and Privacy*, 2023, pp. 2499–2516. <https://doi.org/10.1109/SP46215.2023.10179346>
- [4] Z. Chen, S. M. Beillahi, and F. Long, “FlashSyn: Flash Loan Attack Synthesis via Counter Example Driven Approximation,” *IEEE/ACM International Conference on Software Engineering*, 2024. <https://doi.org/10.1145/3597503.3639190>
- [5] H. Wen, H. Liu, J. Song, Y. Chen, W. Guo, and Y. Feng, “FORAY: Towards Effective Attack Synthesis against Deep Logical Vulnerabilities in DeFi Protocols,” *ACM CCS*, 2024, pp. 1001–1015. <https://doi.org/10.1145/3658644.3690293>
- [6] D. Pusceddu and M. Bartoletti, “Formalizing Automated Market Makers in the Lean 4 Theorem Prover,” *FMBC*, 2024. <https://doi.org/10.4230/OASlcs.FMBC.2024.5>
- [7] K. Dave, V. Sjöberg, and X. Sun, “Towards Verified Price Oracles for Decentralized Exchange Protocols,” *FMBC*, 2021. <https://doi.org/10.4230/OASlcs.FMBC.2021.1>

- [8] G. Angeris and T. Chitra, “Improved Price Oracles: Constant Function Market Makers,” ACM AFT, 2020, pp. 80–91.
<https://doi.org/10.1145/3419614.3423251>
- [9] L. de Moura and N. Bjørner, “Z3: An Efficient SMT Solver,” TACAS, 2008, pp. 337–340. https://doi.org/10.1007/978-3-540-78800-3_24
- [10] A. Meurer et al., “SymPy: Symbolic Computing in Python,” PeerJ Computer Science, vol. 3, e103, 2017. <https://doi.org/10.7717/peerj-cs.103>
- [11] H. Adams et al., “Uniswap v3 Core,” 2021.
<https://uniswap.org/whitepaper-v3.pdf>
- [12] Harvest Finance, “fUSDC/fUSDT Economic Attack Oct 26 2020.”
<https://docs.harvest.finance/other/security/incidents/fusdc-fusdt-economic-attack-oct-26-2020>
- [13] Euler Finance, “War & Peace: Behind the Scenes of Euler’s \$240M Exploit Recovery.” <https://www.euler.finance/blog/war-peace-behind-the-scenes-of-eulers-240m-exploit-recovery>
- [14] Z. Chen, S. M. Beillahi, and F. Long, “FlashSyn Artifact,” ICSE 2024.
<https://github.com/FlashSyn-Artifact/FlashSyn-Artifact-ICSE24>